DATABASE DESIGN ARCHITECTURE

Local SQLite Relational Schema & ERD Specification

Database structure, 3NF normalization rules, Entity-Relationship logic, and DDL schema for an offline-first nutrition app.

1. Database Design Overview & Principles

The database layer is engineered as a local embedded **SQLite database** managed via **Drizzle ORM** on Expo (`expo-sqlite`). It follows **3rd Normal Form (3NF)** to eliminate macro redundancy, guarantee offline search resilience, and ensure zero-latency aggregation for daily analytics.

2. Entity-Relationship Diagram (ERD)

```
+------------------+ +------------------+ +------------------+ | user_profile | | weight_logs |
```

3. Database Normalization Analysis (3NF Compliance)

1NF (First Normal Form): All column values are atomic (e.g., individual numeric macro metrics, single-date ISO strings). No repeating groups or comma-separated lists are stored.

2NF (Second Normal Form): All non-key attributes in `food_logs`, `weight_logs`, `foods`, and `exercise_logs` depend entirely on their primary keys (`id`).

3NF (Third Normal Form): Transitive dependencies are removed. Food nutrition metrics (`calories`, `protein`, `carbs`, `fat`) belong strictly to the `foods` table. The `food_logs` table references `food_id` and records only consumption quantity (`servings_consumed`), preventing data duplication across logs.

4. Data Dictionary & Schema Specifications

Table Name	Primary / Foreign Keys	Attributes & Constraints	Purpose
`user_profile`	PK: `id` (Check id = 1)	name, age, weight, height, gender, activity_level, goal_type, calorie_target, profile_picture_uri	Stores single-user metrics & calculated TDEE.
`foods`	PK: `id` (UUID / API Key)	name, brand, source (usda/fatsecret/custom), serving_size, serving_unit, calories, protein, carbs, fat	Master food catalog & local offline API cache.
`food_logs`	PK: `id` FK: `food_id` → `foods.id`	meal_type (breakfast/lunch/dinner/snack), servings_consumed, logged_date (YYYY-MM-DD), logged_at	Normalized user meal logs.
`weight_logs`	PK: `id` (UUID)	weight (kg), logged_date (YYYY-MM-DD), notes	Historical weight & weigh-in prompts.
`exercise_logs`	PK: `id` (UUID)	source (strava/manual), activity_name, calories_burned, logged_date (YYYY-MM-DD), synced_at	Strava & workout energy burn tracking.

5. Complete SQLite DDL Script

```
CREATE TABLE IF NOT EXISTS user_profile ( id INTEGER PRIMARY KEY CHECK (id = 1), name TEXT NOT NULL, age INTEGER NOT NULL, weight REAL NOT NULL, height REAL NOT NULL, gender TEXT NOT NULL, activity_level REAL NOT NULL, goal_type TEXT NOT NULL, calorie_target INTEGER NOT NULL, profile_picture_uri TEXT, updated_at TEXT NOT NULL ); CREATE TABLE IF NOT EXISTS foods ( id TEXT PRIMARY KEY, name TEXT NOT NULL, brand TEXT, source TEXT NOT NULL, serving_size REAL NOT NULL, serving_unit TEXT NOT NULL, calories INTEGER NOT NULL, protein REAL NOT NULL, carbs REAL NOT NULL, fat REAL NOT NULL, created_at TEXT NOT NULL ); CREATE TABLE IF NOT EXISTS food_logs ( id TEXT PRIMARY KEY, food_id TEXT NOT NULL, meal_type TEXT NOT NULL, servings_consumed REAL NOT NULL, logged_date TEXT NOT NULL, logged_at TEXT NOT NULL, FOREIGN KEY (food_id) REFERENCES foods(id) ON DELETE CASCADE ); CREATE TABLE IF NOT EXISTS weight_logs ( id TEXT PRIMARY KEY, weight REAL NOT NULL, logged_date TEXT NOT NULL, notes TEXT ); CREATE TABLE IF NOT EXISTS exercise_logs ( id TEXT PRIMARY KEY, source TEXT NOT NULL, activity_name TEXT NOT NULL, calories_burned INTEGER NOT NULL, logged_date TEXT NOT NULL, synced_at TEXT NOT NULL );
```